Auditoria de Segurança Autorizada — velnox.app.br

Data: 2026-08-20

Tipo: Red Team controlado, não-destrutivo, autorizado pelo proprietário

Escopo: código, arquitetura, endpoints, autenticação, autorização, isolamento multi-tenant, painel, banco, pagamento (teste), uploads, sessões, recuperação de senha, lógica de negócio.

Resumo

A postura é forte. O isolamento multi-tenant está correto em toda a superfície analisada, não há IDOR, não há escalação de privilégio, o preço do checkout é recalculado no servidor e o fluxo do entregador é rigorosamente escopado. As falhas encontradas são de **gestão de sessão e de abuso de endpoints públicos** — reais, mas de severidade média/baixa. Nenhuma permite vazamento entre restaurantes ou acesso não autorizado a dados de outro tenant.

O hardening da auditoria anterior (mesmo dia) já está em produção: rate limiting, cabeçalhos de segurança, bcrypt custo 12, correção da dependência vulnerável e **rotação da senha do banco** (as senhas que vazaram foram invalidadas).

Vulnerabilidade 1 — Sessão não é revogada após reset de senha

- **Severidade:** MÉDIA
- **Componente:** Autenticação

IMPACTO

Um atacante que já tenha uma sessão ativa (cookie roubado, dispositivo compartilhado, sessão aberta antes do incidente) continua com acesso por até 7 dias mesmo depois de o dono redefinir a senha. A troca de senha, que é a ação de recuperação de conta comprometida, não expulsa o invasor.

EVIDÊNCIA

`src/server/auth/session.ts` — a sessão guarda só `adminUserId` num cookie selado (stateless), sem versionamento. `src/server/services/password-reset-service.ts:155` troca o `passwordHash` mas não invalida nada:

```
await tx.adminUser.update({ where: { id: pedido.adminUserId }, data: { passwordHash } });
// nenhuma invalidação de sessão existente
```

Não há campo `sessionVersion` / `tokenVersion` nem store de sessão server-side. `logoutAdmin` limpa o cookie só do dispositivo atual.

CENÁRIO (NÃO-DESTRUTIVO)

O atacante obtém um cookie de sessão de um dono (num PC compartilhado da lanchonete, por exemplo). O dono percebe algo estranho e redefine a senha pelo e-mail. O cookie do atacante continua válido — ele segue vendo pedidos, clientes e faturamento até o cookie expirar sozinho.

CAUSA RAIZ

Sessões stateless seladas, sem um contador de versão que a validação compare com o banco. Sem esse elo, uma troca de senha não tem como "avisar" as sessões antigas.

CORREÇÃO

```
model AdminUser {
  // ...
  sessionVersion Int @default(0)
}
```

```
// no cookie, guardar também a versão:
await updateSession(sessionConfig(), { adminUserId, sessionVersion: admin.sessionVersion });

// em getCurrentAdmin, comparar:
if (admin.sessionVersion !== session.data.sessionVersion) return null;

// em redefinirSenha (e em setCourierActive/delete), dentro da transação:
await tx.adminUser.update({
  where: { id },
  data: { passwordHash, sessionVersion: { increment: 1 } },
});
```

Isso derruba toda sessão antiga no reset. Custa uma coluna e uma comparação por request.

TESTE DE REGRESSÃO

Logar em dois navegadores; redefinir a senha em um; confirmar que o outro cai para a tela de login no próximo request.

Vulnerabilidade 2 — `visitorId` controlado pelo cliente permite burlar "um por aparelho"

- **Severidade:** BAIXA
- **Componente:** Lógica de negócio

IMPACTO

As regras "uma avaliação por aparelho" e "um like por visitante" são dedupadas por um `visitorId` que o próprio navegador gera e envia. Gerando ids novos, um usuário posta avaliações e infla likes à vontade — manipulação de reputação da loja.

EVIDÊNCIA

`src/server/functions/reviews.fn.ts` / `post-likes.fn.ts` — `visitorId` é `z.string().trim().min(8).max(64)` vindo do payload. `submitReviewFn` ganhou rate limit (5/min), mas `toggleReviewLikeFn` e `togglePostLikeFn` não têm nenhum — inflar likes é ilimitado por origem.

CENÁRIO

Script que, para cada avaliação da loja-alvo, chama `toggleReviewLikeFn` com um `visitorId` aleatório novo → contador de likes sobe sem teto.

CAUSA RAIZ

Identidade de visitante é um segredo compartilhado com o cliente, não derivada de nada server-side. É inevitável sem login do cliente; o que dá para conter é o volume.

CORREÇÃO

Aplicar `permitido(...)` também nos dois toggles de like (ex.: 15/min por origem) e, opcionalmente, ligar o `visitorId` a um hash do IP+User-Agent no servidor para encarecer a fabricação. Não elimina, mas torna o abuso caro.

TESTE DE REGRESSÃO

Disparar 30 likes em 1 min da mesma origem e confirmar bloqueio a partir do 16º.

Vulnerabilidade 3 — Enumeração de cupons (preview público sem rate limit)

- **Severidade:** BAIXA
- **Componente:** API / Lógica de negócio

IMPACTO

`previewCouponFn` é público e informa se um código de cupom é válido e qual desconto dá, para uma loja. Sem rate limit, dá para varrer códigos e descobrir cupons não divulgados (ex.: um `VIP50` interno).

EVIDÊNCIA

`src/server/functions/coupons.fn.ts` — `previewCouponFn` está na lista dos 16 endpoints públicos e não recebeu `permitido(...)` no passe de hardening (só login, checkout, review-submit, reset, signup e convite ganharam).

CENÁRIO

Dicionário de códigos comuns (`PROMO10` , `DESCONTO20` , `BLACKFRIDAY`) chamado em rajada contra o preview até um responder com desconto válido.

CAUSA RAIZ

Endpoint de verificação sem teto de tentativas — o mesmo padrão dos outros públicos, só que este passou batido.

CORREÇÃO

```
if (!permitido("coupon-preview", { max: 20, janelaMs: 60_000 })) {
  return { ok: false as const, error: "Muitas tentativas. Aguarde um instante." };
}
```

no início do handler.

TESTE DE REGRESSÃO

25 previews em 1 min da mesma origem → bloqueio a partir do 21º.

Vulnerabilidade 4 — Conteúdo do arquivo enviado não é validado (só o content-type é fixado)

- **Severidade:** BAIXA
- **Componente:** Uploads

IMPACTO

O presign valida o `contentType` contra uma allowlist e fixa ele no objeto (`ContentType` assinado no `PutObjectCommand`), mas os bytes não são inspecionados — dá para subir um arquivo arbitrário rotulado como `image/jpeg`. Risco real é baixo porque os arquivos são servidos de um domínio separado (`cdn.velnox.app.br`) com content-type pinado, e um `<img>` não executa um polyglot.

EVIDÊNCIA

`src/server/storage/presign.ts` — `ALLOWED_TYPES` cobre o rótulo; o corpo não é verificado (nem magic bytes).

CORREÇÃO (DEFESA EM PROFUNDIDADE)

Validar os magic bytes no `/api/upload` local (fallback dev) e, no R2, confiar no content-type pinado + garantir que o bucket não serve `Content-Disposition: inline` para tipos inesperados. Confirmar que o bucket R2 está com listagem pública desabilitada (as chaves usam UUID, então não são enumeráveis, mas listagem aberta anularia isso).

TESTE DE REGRESSÃO

Subir um `.html` renomeado com content-type `image/jpeg` e confirmar que o CDN o entrega como imagem (não executável) e que `GET` na raiz do bucket não lista objetos.

Informativo — Sem verificação de e-mail no cadastro

- **Severidade:** INFORMATIVA
- **Componente:** Autenticação

`createStoreAccountFn` cria o dono e abre a sessão na hora, sem confirmar o e-mail. Não é falha de isolamento (a conta nova não acessa dados de ninguém), mas permite cadastrar com um e-mail que não se controla. Como o reset de senha vai para esse e-mail, o ideal é verificar antes de considerar a conta "de verdade".
Baixa prioridade.

O que foi testado e está CORRETO (não mexer)

Área	Verificação
Isolamento multi-tenant	Toda consulta e mutação de painel filtra por `storeId`; mutações por id são precedidas de `findFirst({id, storeId})` no mesmo request
BOLA/IDOR	Nenhum. Slug/id de outra loja simplesmente "não encontrado"
Escalação vertical	`COURIER` barrado do painel; billing exige `OWNER`; `MANAGER` nem é criável
Perfil 3→B (tenant cruzado)	Reviews via `{id, storeId}` (P2025 se cruzar); entregador só fecha entrega `{id, storeId, courierId}` dele
Checkout	Preço recalculado no servidor; ids validados por loja; `client_reference_id` vem da sessão, não do cliente
Webhook Stripe	Assinatura verificada; loja ligada por `metadata.storeId` / `stripeCustomerId` server-side
Sessão	Relê `isActive` do banco a cada request (conta desativada perde acesso na hora); cookie httpOnly+secure+sameSite
Uploads	`storeId` da sessão; chave `stores/<id>/<uuid>.<ext>`; path traversal barrado por regex; allowlist de content-type; limite 2MB
Token de rastreio	192 bits (CSPRNG); resposta mínima (nome que o próprio cliente digitou; sem telefone, endereço ou notas internas); token errado e pedido apagado respondem igual

Priorização

Prioridade	Vulnerabilidade	Severidade	Impacto	Correção
1	Sessão não revogada no reset de senha	MÉDIA	Invasor com sessão sobrevive à recuperação de conta	Coluna `sessionVersion` + `increment` no reset
2	Cupons enumeráveis (preview sem rate limit)	BAIXA	Descoberta de cupons secretos	`permitido("coupon-preview", …)`
3	`visitorId` client-side (likes sem rate limit)	BAIXA	Inflar reputação da loja	Rate limit nos toggles de like
4	Bytes de upload não validados	BAIXA	Baixo (CDN cross-origin + type pinado)	Magic bytes + confirmar bucket sem listagem
5	Cadastro sem verificação de e-mail	INFORMATIVA	Squatting de e-mail	Fluxo de verificação (futuro)

Qual é o maior risco de segurança encontrado no sistema?

A ausência de revogação de sessão (Vulnerabilidade 1). É o único achado que quebra uma garantia que o usuário espera existir: "troquei minha senha, logo expulsei quem estava dentro". Hoje isso não acontece — um invasor com sessão ativa permanece por até 7 dias após a recuperação. Todo o resto do sistema é sólido justamente na parte mais difícil (isolamento entre lojas), então esse elo de sessão vira o ponto mais fraco por contraste.

Qual correção deveria ser implementada primeiro?

A coluna `sessionVersion` com incremento no reset de senha. É de baixo custo (uma coluna, uma comparação por request, um `increment` na transação que já existe) e fecha o único caminho que sobrevive a uma recuperação de conta. As outras três são todas rate-limiting/hardening de endpoint público — mecânicas e rápidas, mas de impacto menor.

Estado das correções

☐ 1. `sessionVersion` no reset de senha (MÉDIA)
☐ 2. Rate limit no `previewCouponFn` (BAIXA)
☐ 3. Rate limit nos toggles de like (BAIXA)
☐ 4. Validação de magic bytes + bucket sem listagem (BAIXA)
☐ 5. Verificação de e-mail no cadastro (INFORMATIVA)

Correções 1, 2 e 3 são código de baixo risco e testável, prontas para aplicar.